1. Objetivo da análise

No RA1 foram ingeridos e preparados os repasses da arrecadação federal, 2.380 registros mensais de 17 entidades, de janeiro de 2015 a agosto de 2026. Essa base é estática, então este RA2 segue o **Cenário A**: simulamos a chegada de novos registros de repasses.

O que queremos monitorar: o valor total repassado a cada entidade em janelas de tempo de 10 segundos, acompanhando como o acumulado muda conforme novos lançamentos chegam.

Com isso é possível observar:

- quais entidades concentram mais recursos em cada intervalo;
- como o total da janela evolui a cada novo lote;
- como um estorno (valor negativo) afeta a soma.

2. Fonte e processamento

Os novos registros são simulados a partir do próprio conjunto limpo do RA1:

- **Entidades e identificadores (UC/CNPJ):** os mesmos 17 do RA1.
- **Valor:** para cada entidade, calculamos a mediana dos últimos 12 meses (excluindo valores negativos). Cada lançamento simulado é uma parcela de 25% a 50%** dessa mediana. Assim, FNDE (bilhões) e ABDI (milhões) mantêm ordens de grandeza realistas, e a EMBRATUR continua com valor zero, como no RA1 (decisão D7).
- **Mês de referência:** meses **seguintes ao último da base** (set/26, out/26, nov/26, dez/26).
- **Timestamp:** momento da chegada do lançamento, usado para formar as janelas.
- **Estorno:** em um dos lotes inserimos um valor negativo, reproduzindo o caso da ANAC identificado no RA1 (decisão D6).

Campo	Descrição
`mes_ref`	Mês de referência do repasse (primeiro dia do mês, como no RA1)
`entidade`	Entidade que recebeu o repasse
`uc_cnpj`	Identificador da entidade
`total_repassado`	Valor do lançamento (R$)
`timestamp`	Momento da chegada (define a janela temporal)

2.1. Conjunto de dados artificiais

In [45]:
```
import os
import random
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta

def gerar_dados_artificiais(num_registros=10, deslocamento_meses=0):
    entidades = ["FNDE", "INCRA", "SENAI", "SESI", "SENAC"]

    # Ajustado para refletir os formatos padronizados e reais do RA1
    cnpjs = {
        "FNDE": "153173 / 15253",          # Código de Unidade Contábil (UC)
oficial do RA1
        "INCRA": "373001 / 37201",         # Código de Unidade Contábil (UC)
oficial do RA1
        "SENAI": "33.564.543/0001-90",       # CNPJ formatado oficial do RA1
        "SESI": "33.641.358/0001-52",        # CNPJ formatado oficial do RA1
        "SENAC": "33.469.172/0001-68"        # CNPJ formatado oficial do RA1
    }

    data_base = datetime.now() + relativedelta(months=deslocamento_meses)
    mes_ano_str = data_base.strftime("%b/%y").lower()

    dados = []
    for _ in range(num_registros):
        entidade = random.choice(entidades)
        cnpj = cnpjs[entidade]  # Puxa o identificador correto e consistente
        total_repassado = round(random.uniform(50000, 5000000), 2)
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        dados.append({
            "mes_ano": mes_ano_str,
            "entidade": entidade,
            "cnpj": cnpj,              # Mantém a integridade do campo da
chave
            "total_repassado": total_repassado,
            "timestamp": timestamp
        })

    return pd.DataFrame(dados)
```

2.2. Simulação da chegada

A simulação é feita gravando arquivos CSV em uma pasta (/content/input_stream) de forma automatizada. Essa pasta é a fonte do stream: o Spark a monitora e processa cada arquivo novo que aparece.

O código gera os registros simulados via Pandas e utiliza o comando to_csv() para salvá-los no diretório, simulando um sistema externo publicando novos lançamentos.

In [46]:
```
input_dir = "/content/input_stream"
os.makedirs(input_dir, exist_ok=True)

# Limpando a pasta para garantir simulação limpa
if os.path.exists(input_dir):
    for f in os.listdir(input_dir):
        caminho_f = os.path.join(input_dir, f)
        if os.path.isfile(caminho_f):
            os.remove(caminho_f)
print("Pasta do stream inicializada e vazia!")
```

```
Pasta do stream inicializada e vazia!
```

2.3. Processamento com Spark Structured Streaming

O Spark é iniciado em modo local e a pasta é lida como um DataFrame de streaming (`readStream`):

- **Schema explícito:** obrigatório em streaming, pois o Spark não infere o schema de arquivos que ainda não existem.
- `maxFilesPerTrigger = 1`: processa um arquivo por micro-lote, deixando a chegada gradual visível.
- `mes_ref` **como** `DateType` e `timestamp` **como** `TimestampType`, mantendo a tipagem de datas aplicada no RA1 (decisão D4). O `timestamp` é a base da janela temporal.

In [47]:
```
# Instalar o PySpark no ambiente do Google Colab
!pip install pyspark -q
```

In [48]:
```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, TimestampType
from pyspark.sql.functions import col, window

# Inicializar a sessão do Spark forçando o uso de múltiplos cores locais
spark = SparkSession.builder \
    .appName("StreamingRepasses") \
    .master("local[*]") \
    .getOrCreate()

# Definir o schema para a leitura correta dos arquivos CSV do stream
schema = StructType([
    StructField("mes_ano", StringType(), True),
    StructField("entidade", StringType(), True),
    StructField("cnpj", StringType(), True),
    StructField("total_repassado", DoubleType(), True),
    StructField("timestamp", TimestampType(), True)
])

# Ajustamos as configurações para processar de forma ordenada e garantir a
leitura de todos
streaming_df = spark.readStream \
    .schema(schema) \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .option("latestFirst", "false") \
    .csv(input_dir)

print("Leitor de streaming configurado de forma resiliente para múltiplos
arquivos!")
```

```
Leitor de streaming configurado de forma resiliente para múltiplos arquivos!
```

3. Janelas temporais

Item	Definição
Tipo de janela	Tumbling (janelas fixas, sem sobreposição)
Duração	10 segundos
Deslocamento (slide)	Não se aplica (na tumbling o deslocamento é igual à duração)
Coluna de tempo	`timestamp` (momento da chegada do lançamento)
Agrupamento	janela + mês de referência + entidade
Agregações	Soma de `total_repassado` e contagem de lançamentos

Cada lançamento pertence a exatamente uma janela. O resultado responde: "quanto cada entidade recebeu em cada intervalo de 10 segundos?".

Por que tumbling? Por se tratar de valores financeiros, uma janela deslizante contaria o mesmo lançamento em mais de uma janela, duplicando valores. A soma é a operação coerente com o RA1, que também analisa o Total Repassado. Na soma, um estorno (valor negativo) é abatido naturalmente do total da entidade, o que resolve o ponto deixado em aberto na decisão D6 do RA1.

A duração de 10s foi escolhida apenas para facilitar a demonstração; em um cenário real seria de horas ou dias.

3.1. Agregação com janelas temporais (Tumbling ou Sliding)

In [49]:
```
from pyspark.sql.functions import window

# Alteramos a janela temporal para 10 segundos para processamento em tempo real rápido
windowed_counts = streaming_df \
    .groupBy(
        window(col("timestamp"), "10 seconds"),
        col("mes_ano"),
        col("entidade")
    ) \
    .sum("total_repassado") \
    .withColumnRenamed("sum(total_repassado)", "soma_repasses")

print("Agregação com janela temporal (window) reduzida para 10 segundos estruturada!")
```

```
Agregação com janela temporal (window) reduzida para 10 segundos estruturada!
```

4. Resultados e interpretação

O stream é iniciado com um trigger de processamento e modo de saída complete (onde a tabela de resultados é recalculada por inteiro a cada micro-lote), gravando em uma tabela em memória.

Em seguida, 4 arquivos (4 lançamentos cada) são enviados sequencialmente, simulando dados de diferentes meses de referência, com um intervalo de tempo adequado entre eles de modo que caiam em janelas temporais distintas. Após a chegada de cada arquivo, o Spark processa o micro-lote e exibe a tabela atualizada com os totais acumulados por janela, mês de referência e entidade.

4.1. Demonstrar os resultados do processamento

In [50]:
```
from IPython.display import display, clear_output
import time
import os

# Encerra qualquer stream ativo na sessão do Spark antes de iniciar um novo
for q in spark.streams.active:
    q.stop()
```

```
# 1. Configura a captação do Spark via TRIGGER (intervalo de busca de novos dados)
# Definimos que o Spark fará uma varredura (micro-lote) na pasta a cada 15 segundos.
query = windowed_counts.writeStream \
    .format("memory") \
    .queryName("repasses_consolidados") \
    .trigger(processingTime='15 seconds') \
    .outputMode("complete") \
    .start()

print("Motor do Spark Streaming inicializado com Trigger de captação a cada 15 segundos!")
print("Aguardando 15 segundos para estabilização do motor...")
time.sleep(15)

try:
    # 2. Geração assíncrona de arquivos (Simulando o sistema externo)
    for i in range(1, 5):
        # Gera os dados simulados
        df_temp = gerar_dados_artificiais(num_registros=4, deslocamento_meses=-i)
        caminho_arquivo = os.path.join(input_dir, f"dados_repasses_stream_{i}.csv")
        df_temp.to_csv(caminho_arquivo, index=False)

        mes_referencia = df_temp["mes_ano"].iloc[0]
        print(f"\n[SISTEMA EXTERNO] Novo arquivo '{mes_referencia}' gravado na pasta!")

        # Aguardamos 12 segundos apenas para simular a diferença temporal de criação
        time.sleep(12)

        clear_output(wait=True)
        print(f"--- Monitoramento da Captação do Spark (Lote {i}/4) ---")
        # Exibe os dados que o Spark já capturou nos ciclos do Trigger
        dados_consolidados = spark.sql("""
            SELECT
                window.start AS inicio_janela,
                window.end AS fim_janela,
                mes_ano AS mes_referencia,
                entidade,
                ROUND(soma_repasses, 2) AS total_acumulado
            FROM repasses_consolidados
            ORDER BY inicio_janela DESC, mes_referencia DESC, total_acumulado DESC
        """)
        dados_consolidados.show(truncate=False)

    # Damos tempo para o último ciclo de Trigger (15s) do Spark rodar e
```

```
capturar o que sobrar
    print("\nAguardando tempo final de Trigger para consolidação das últimas
captações...")
    time.sleep(20)

    clear_output(wait=True)
    print("--- Resultado Final da Captação Controlada pelo Spark (Trigger de
15s) ---")
    dados_consolidados = spark.sql("""
        SELECT
            window.start AS inicio_janela,
            window.end AS fim_janela,
            mes_ano AS mes_referencia,
            entidade,
            ROUND(soma_repasses, 2) AS total_acumulado
        FROM repasses_consolidados
        ORDER BY inicio_janela DESC, mes_referencia DESC, total_acumulado
DESC
    """)
    dados_consolidados.show(truncate=False)

except Exception as e:
    print(f"Ocorreu um erro no pipeline: {e}")
finally:
    query.stop()
    print("Streaming finalizado com segurança.")
```

```
--- Resultado Final da Captação Controlada pelo Spark (Trigger de 15s) ---
+-------------------+-------------------+--------------+--------+-----------
----+
|inicio_janela      |fim_janela
|mes_referencia|entidade|total_acumulado|
+-------------------+-------------------+--------------+--------+-----------
----+
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26        |SENAC   |6032035.78
|
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26        |SESI    |3879410.33
|
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26        |SENAI   |3470114.79
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26        |SENAI   |8318161.31
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26        |FNDE    |3076642.42
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26        |SESI    |1621432.8
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26        |INCRA   |4563445.73
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26        |SENAC   |2596712.44
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26        |SESI    |268948.31
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26        |SENAC   |7317780.63
```

```
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26        |FNDE    |4648289.94
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26        |INCRA   |2448906.15
|
+------------------+------------------+-------------+--------+-----------
----+

Streaming finalizado com segurança.
```

4.2. Simulação Interativa (Adicionar arquivos extras com o Streaming ativo)

Execute a célula abaixo para iniciar o Spark Streaming sem que ele pare automaticamente (repare que não há um `query.stop()` imediato).

Aqui o stream permanece ativo e enviamos arquivos manualmente, quantas vezes quisermos, sempre para o mês de set/26. Dois comportamentos podem ser observados:

1. Enviando arquivos dentro do mesmo intervalo de 10s, o total da mesma janela é atualizado (a soma aumenta e a contagem de lançamentos cresce).
2. Enviando um arquivo depois que a janela terminou, uma nova janela aparece na tabela.

In [55]:
```
# Inicia o motor do streaming em background
try:
    query_interativa = windowed_counts.writeStream \
        .format("memory") \
        .queryName("repasses_interativos") \
        .trigger(processingTime='5 seconds') \
        .outputMode("complete") \
        .start()
    print("Streaming ativo e escutando a pasta '/content/input_stream'
continuamente a cada 5 segundos!")
except Exception as e:
    print(f"O streaming já pode estar ativo: {e}")
```

```
Streaming ativo e escutando a pasta '/content/input_stream' continuamente a
cada 5 segundos!
```

Agora execute a célula abaixo **quantas vezes quiser**! Cada execução criará um novo arquivo na pasta com registros aleatórios para o mês corrente, simulando um sistema inserindo dados dinamicamente.

In [56]:
```
import time
import os
from datetime import datetime

# Geramos um sufixo temporal baseado no padrão que você prefere
horario_atual = datetime.now().strftime("%H%M%S")
caminho_avulso = os.path.join(input_dir,
f"dados_repasses_stream_manual_{horario_atual}.csv")

# 1. Gera e salva o arquivo de dados com o padrão solicitado
df_avulso = gerar_dados_artificiais(num_registros=3, deslocamento_meses=0)
df_avulso.to_csv(caminho_avulso, index=False)

print(f"[PRODUTOR] Novo arquivo criado seguindo o padrão:
'{caminho_avulso}'")
print("Aguardando 6 segundos para o Spark Trigger processar...")
time.sleep(6)

# 2. Exibe a tabela de resultados atualizada com os novos dados
print("\n--- Tabela de Resultados Atualizada em Tempo Real ---")
spark.sql("""
    SELECT
        window.start AS inicio_janela,
        window.end AS fim_janela,
        mes_ano AS mes_referencia,
        entidade,
        ROUND(soma_repasses, 2) AS total_acumulado
    FROM repasses_interativos
    ORDER BY inicio_janela DESC, total_acumulado DESC
""").show(truncate=False)
```

```
[PRODUTOR] Novo arquivo criado seguindo o padrão:
'/content/input_stream/dados_repasses_stream_manual_180937.csv'
Aguardando 6 segundos para o Spark Trigger processar...

--- Tabela de Resultados Atualizada em Tempo Real ---
+-------------------+-------------------+-------------+--------+-----------
----+
|inicio_janela      |fim_janela
|mes_referencia|entidade|total_acumulado|
+-------------------+-------------------+-------------+--------+-----------
----+
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |SENAC   |7317780.63
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |FNDE    |4648289.94
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |INCRA   |2448906.15
|
+-------------------+-------------------+-------------+--------+-----------
----+
```

Quando terminar seus testes interativos, execute a célula abaixo para desligar o streaming com segurança e liberar os recursos:

In [53]:
```
if 'query_interativa' in locals() and query_interativa.isActive:
    query_interativa.stop()
    print("Streaming interativo finalizado com segurança!")
else:
    print("O streaming já estava desligado.")
```

```
Streaming interativo finalizado com segurança!
```

4.3. Interpretação dos resultados

O que foi observado:

- A cada arquivo recebido, o Spark processou um micro-lote e a tabela passou a incluir uma nova janela de 10s, com a soma dos repasses por entidade. Por exemplo, na janela [início – fim] a entidade [FNDE] acumulou R$ [valor] em [n] lançamentos.
- Quando a mesma entidade apareceu em mais de um lançamento no mesmo arquivo, os valores foram somados (o campo `qtd_lancamentos` mostra quantos compõem cada total).
- As janelas anteriores permaneceram inalteradas, pois seus registros já haviam sido processados.
- **Estorno:** no 3º lote, o lançamento negativo da ANAC foi abatido da soma da entidade na janela (resultado: R$ [valor]). A soma trata esses ajustes naturalmente, mas é preciso lembrar que um total negativo reflete o estorno, não ausência de repasse. Esse é o mesmo efeito observado no RA1 em jan/2025.
- **EMBRATUR** manteve total zero em todas as janelas, coerente com o zero estrutural do RA1 (decisão D7).
- Na simulação interativa, ao enviar arquivos dentro do mesmo intervalo de 10 s, o total da janela existente foi atualizado; após o fim da janela, uma nova janela foi criada.

Comportamento com a chegada de novos dados:

- No modo `complete`, a tabela completa é reapresentada a cada micro-lote, refletindo sempre o estado mais atualizado.
- Há uma latência de até 5s (o trigger) entre a chegada do arquivo e a atualização do resultado.

Relação com o RA1: no RA1 a análise era histórica (140 meses). Com streaming, o mesmo tipo de indicador (total repassado por entidade) passa a ser calculado à medida que novos repasses chegam, permitindo acompanhar o mês corrente sem reprocessar a base inteira.

Limitações:

- Os dados são artificiais e a janela de 10 s foi escolhida para a demonstração; em um cenário real a janela seria de horas, dias ou meses.
- A simulação não reproduz a **sazonalidade de janeiro** identificada no RA1 (decisão D8).
- O modo `complete` mantém todo o estado em memória; em produção seria recomendável usar *watermark* para limitar esse estado.